Análisis y Diseño – Sistema de Pedidos Anticipados Cafetería UdeC Ubaté

Nov 10, 2026 · @michell daniela caicedo martinez

1. Introducción y objetivos

El sistema permite que los estudiantes de la UdeC seccional Ubaté vean el menú y pidan su almuerzo antes de llegar a la cafetería, para recogerlo en una franja de 15 minutos sin hacer fila. Responde al problema identificado en el *Diagnóstico Binario* (6 de noviembre de 2026): el 93 % de los encuestados ha llegado tarde a clase por esperar en la cafetería, el 79 % se queda sin almorzar al menos a veces y el 86 % solo conoce el menú al llegar. El 79 % afirmó que usaría una aplicación web para pedir con anticipación.

Este documento cubre las etapas de **análisis** (completar los requisitos) y **diseño** (definir cómo se construye) que van antes del desarrollo. Parte de las 15 historias de usuario y del product backlog ya elaborados.

Objetivo general. Diseñar un sistema web basado en dos microservicios, desplegados con Docker, que permita publicar el menú del día y recibir pedidos anticipados con cupo limitado por franja horaria en la cafetería de la UdeC seccional Ubaté.

Objetivos específicos.

1. Especificar los requisitos funcionales y no funcionales a partir de las historias de usuario.
2. Definir la arquitectura de los microservicios **menu-ms** y **orders-ms** y la forma en que se comunican por API REST.
3. Diseñar el modelo de datos, los endpoints y las pantallas del sistema.
4. Planear las pruebas, el despliegue con Docker Compose y el control de versiones con Git.

2. Alcance y limitaciones

El sistema cubre el ciclo completo de un pedido anticipado: desde que el encargado publica el menú hasta que entrega el almuerzo y registra el pago. No reemplaza la fila tradicional; la complementa con una caja de pedidos anticipados.

Incluido	No incluido (limitaciones)
Registro de productos, precios, estado (Disponible/Agotado) y unidades del día	Pago en línea: el estudiante paga al recoger, en efectivo o transferencia (Nequi, Daviplata)
Consulta del menú del día, con filtro por categoría	Inicio de sesión con cuenta institucional; el estudiante se identifica con nombre y código
Pedido con varios productos, número consecutivo diario y franja de recogida de 15 minutos	Notificaciones por correo, SMS o WhatsApp; el estudiante consulta el estado en la página
Cupo máximo de pedidos por franja	Inventario de ingredientes y compras de la cafetería
Lista de pendientes para el encargado, marcar "Listo", entrega y registro del medio de pago	Aplicación móvil nativa; es una página web que funciona en el celular
Cancelación del pedido mientras esté en estado "Recibido"	Solución al déficit de mesas y espacio, que depende de la administración

Como señaló el diagnóstico, la falta de espacio y los horarios simultáneos de salida no se resuelven con software; el sistema ataca la demora en la atención y el desconocimiento del menú.

3. Actores y casos de uso

El sistema tiene dos actores. El **estudiante** consulta el menú, pide, consulta y cancela su pedido, sin iniciar sesión. El **encargado de la cafetería** administra los productos, prepara y entrega los pedidos y fija el cupo por franja.

El estudiante pide y consulta; el encargado administra el menú y los pedidos



Diagrama de casos de uso · 2 actores, 13 casos de uso

Cada caso de uso corresponde a una o dos historias de usuario (tabla 4.1). El color indica cuál de los dos microservicios lo atiende: seis los resuelve menu-ms (azul) y siete orders-ms (naranja).

4. Requisitos

Los 15 requisitos funcionales salen uno a uno de las historias de usuario; los criterios de aceptación de cada historia son la regla de validación que se prueba (sección 12).

4.1 Requisitos funcionales

ID	Requisito	Historia	Microservicio	Sprint
RF-01	Registrar producto con nombre, categoría y precio mayor a 0, sin nombres duplicados	CF-MENU-0001	menu-ms	1

ID	Requisito	Historia	Microservicio	Sprint
RF-02	Mostrar el menú del día con nombre, categoría, precio y etiqueta "Agotado"	CF-MENU-0002	menu-ms	1
RF-03	Marcar un producto como Agotado o Disponible	CF-MENU-0003	menu-ms	1
RF-04	Hacer un pedido con varios productos, nombre y código, verificando el menú	CF-PEDI-0001	orders-ms	2
RF-05	Asignar número consecutivo diario (001), único aunque haya pedidos simultáneos	CF-PEDI-0002	orders-ms	2
RF-06	Cambiar el precio sin alterar pedidos ya registrados	CF-MENU-0004	menu-ms	2
RF-07	Elegir franja de recogida de 15 minutos; deshabilitar las que empiezan en menos de 15 minutos	CF-PEDI-0003	orders-ms	3
RF-08	Listar pedidos "Recibido" ordenados por franja	CF-PEDI-0004	orders-ms	3
RF-09	Marcar pedido como "Listo"	CF-PEDI-0005	orders-ms	3
RF-10	Consultar el estado de un pedido por su número	CF-PEDI-0006	orders-ms	4
RF-11	Registrar entrega y medio de pago (efectivo o transferencia)	CF-PEDI-0007	orders-ms	4
RF-12	Cancelar pedido solo en estado "Recibido", con confirmación	CF-PEDI-0008	orders-ms	4
RF-13	Registrar unidades del día, descontarlas por pedido y agotar al llegar a 0	CF-MENU-0005	menu-ms	5
RF-14	Definir cupo máximo de pedidos por franja y mostrar franjas llenas	CF-PEDI-0009	orders-ms	5
RF-15	Filtrar el menú por categoría	CF-MENU-0006	menu-ms	5

4.2 Requisitos no funcionales

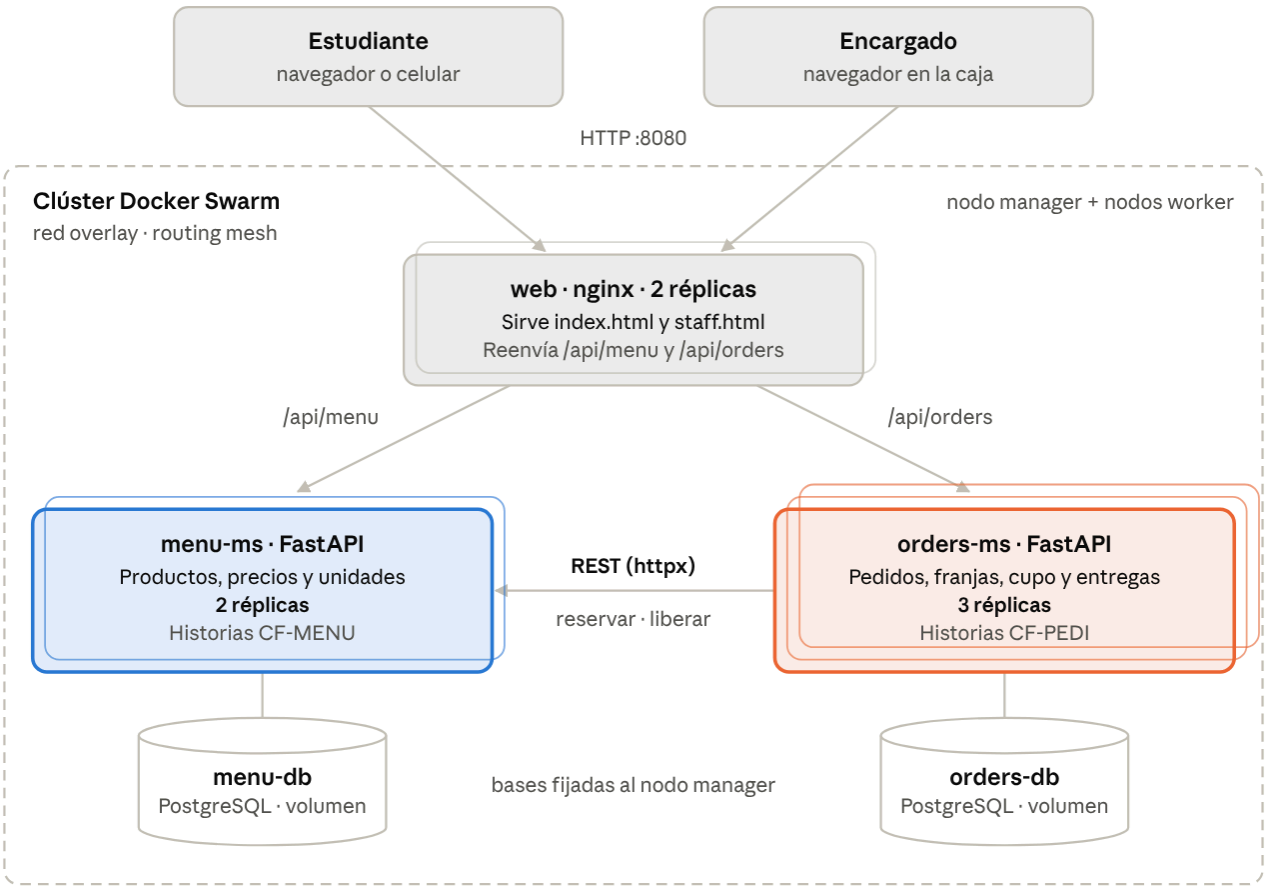
ID	Tipo	Requisito	Cómo se verifica
RNF-01	Usabilidad	La página del estudiante funciona en celular (diseño adaptable desde 360 px de ancho) y un pedido se hace en menos de 1 minuto	Prueba con 3 compañeros desde su celular
RNF-02	Rendimiento	Cada endpoint responde en menos de 1 segundo con 50 pedidos por franja	Medición con Postman o el script de pruebas
RNF-03	Disponibilidad	Si menu-ms no responde, orders-ms no registra el pedido y muestra un mensaje claro (no se cae)	Detener el contenedor menu-ms y probar un pedido
RNF-04	Consistencia	Dos pedidos simultáneos nunca reciben el mismo número ni superan el cupo o las unidades	Prueba de concurrencia automatizada
RNF-05	Portabilidad	Todo el sistema se levanta con un solo comando: `docker compose up --build`	Ejecutarlo en un equipo limpio con Docker
RNF-06	Mantenibilidad	Cada microservicio tiene su propia base de datos, su Dockerfile y pruebas automáticas	Revisión del repositorio
RNF-07	Documentación	Cada API publica su documentación automática en `/docs` (Swagger)	Abrir `/docs` de cada servicio
RNF-08	Idioma	Toda la interfaz y los mensajes están en español, con precios en pesos colombianos (COP)	Revisión de pantallas

5. Arquitectura

El sistema se despliega como un stack de Docker Swarm: una web con nginx (2 réplicas), menu-ms (2 réplicas), orders-ms (3 réplicas, porque recibe la carga de la hora pico) y una base PostgreSQL por microservicio. Si una réplica falla, Swarm la reemplaza y las demás

siguen atendiendo. Los navegadores solo hablan con la web; nginx reenvía cada petición al microservicio que corresponde.

Docker Swarm reparte cada servicio en réplicas; cada uno tiene su propia base



Arquitectura · 3 contenedores en Docker Compose, 2 bases de datos

La única comunicación entre microservicios va de orders-ms a menu-ms: al confirmar un pedido, orders-ms llama a `/products/reserve` para verificar que todo esté disponible y descontar unidades; al cancelar, llama a `/products/release` para devolverlas. Si menu-ms no responde en 3 segundos, el pedido no se registra (CF-PEDI-0001, escenario 4). Si la reserva funciona pero guardar el pedido falla, orders-ms libera las unidades como acción de compensación (patrón saga).

Se separó en dos microservicios porque el menú y los pedidos cambian por razones distintas y los usan personas distintas: el menú lo actualiza el encargado unas pocas veces al día, mientras que los pedidos se concentran en la hora del almuerzo. Así, un fallo en uno no borra los datos del otro y cada uno se puede probar y desplegar por separado.

6. Tecnologías elegidas

Se eligió Python con FastAPI porque produce APIs cortas y legibles, con documentación automática, y el equipo puede explicarlas en la sustentación.

Componente	Tecnología	Por qué
Lenguaje de los microservicios	Python 3.12	Sintaxis sencilla, conocida por el equipo
Framework de API	FastAPI	Crea endpoints REST con validación y documentación Swagger en `/docs` sin código extra
Servidor	Uvicorn	Servidor ASGI recomendado para FastAPI
Base de datos	PostgreSQL 16 (una por microservicio)	Admite escrituras simultáneas de varias réplicas con bloqueos y transacciones; cada base guarda sus datos en un volumen
Comunicación entre servicios	HTTP/REST con JSON (librería httpx)	Es el estándar visto en clase; orders-ms llama a menu-ms
Interfaz web	HTML, CSS y JavaScript	Una página para el estudiante y otra para el encargado, sin frameworks
Servidor web y proxy	Nginx	Sirve la página y reenvía `/api/menu` y `/api/orders` a cada microservicio
Contenedores	Dockerfile, Docker Compose y Docker Swarm	Requisito del proyecto: Dockerfile por servicio, Compose para desarrollar y Swarm para desplegar con réplicas
Control de versiones	Git y GitHub	Requisito del proyecto; historial de cambios por historia de usuario
Pruebas	pytest	Pruebas automáticas de cada criterio de aceptación
Gestión del backlog	Jira o la hoja de Excel del backlog	Seguimiento de los 5 sprints

Componente	Tecnología	Por qué
Desarrollo asistido	Google Antigravity	Agente de IA para generar y revisar el código (sección 14)
Análisis de datos	Google Colab	Análisis y gráficas de la encuesta en Python

7. Modelo de datos

Cada microservicio es dueño de su propia base de datos y ninguno lee la del otro: orders-ms solo conoce los productos a través de la API de menu-ms. Por eso cada pedido guarda una copia del nombre y del precio del producto en el momento de pedir, lo que cumple el escenario 3 de CF-MENU-0004 (los pedidos ya registrados conservan su precio).

7.1 menu-ms — tabla `products`

Campo	Tipo	Regla
id	integer	Llave primaria, autoincremental
name	text	Obligatorio, único sin distinguir mayúsculas
category	text	`lunch`, `drink` o `snack` (en pantalla: almuerzo, bebida o snack)
price	numeric	Obligatorio, mayor a 0 (COP)
status	text	`AVAILABLE` o `SOLD_OUT` (en pantalla: Disponible o Agotado); por defecto `AVAILABLE`
stock	integer	Opcional, mayor o igual a 0; vacío = sin control de unidades

7.2 orders-ms — tablas `orders`, `order_items` y `config`

Tabla	Campo	Tipo	Regla
orders	id	integer	Llave primaria
orders	date	date	Día del pedido; con `number` forma una llave única
orders	number	integer	Consecutivo del día; reinicia en 1 cada día

Tabla	Campo	Tipo	Regla
orders	customer_name, student_code	text	Nombre y código estudiantil, obligatorios
orders	slot	time (HH:MM)	Hora de inicio de la franja de 15 minutos
orders	status	text	`RECEIVED`, `READY`, `DELIVERED` o `CANCELLED` (en pantalla: Recibido, Listo, Entregado, Cancelado)
orders	payment_method	text	`CASH` o `TRANSFER`; se llena al entregar
orders	total	numeric	Suma de precio por cantidad de cada ítem
orders	created_at	timestamp	Momento de confirmación
order_items	order_id	integer	Llave foránea a `orders`
order_items	product_id	integer	Id del producto en menu-ms (sin llave foránea: está en otra base)
order_items	product_name, unit_price	text, numeric	Copia del producto al momento del pedido
order_items	quantity	integer	Mayor a 0
config	key, value	text	`slot_capacity` (por defecto 10), `start_time` (11:30) y `end_time` (14:00)

El horario de franjas de 11:30 a. m. a 2:00 p. m. es una suposición; falta confirmarlo con la cafetería, y queda configurable para no tener que cambiar el código.

8. Diseño de las APIs REST

Las dos APIs reciben y devuelven JSON. Los errores devuelven un código HTTP (400 dato inválido, 404 no existe, 409 conflicto de estado, 503 servicio no disponible) y, en el campo `detail`, el mensaje exacto del criterio de aceptación, para que la página lo muestre tal cual.

8.1 menu-ms (puerto interno 8000)

Método	Ruta	Qué hace	Historia
POST	/products	Registra un producto `{name, category, price}`	CF-MENU-0001
GET	/menu?category=all|lunch|drink|snack	Devuelve el menú del día, filtrado o completo	CF-MENU-0002, 0006
GET	/products/{id}	Consulta un producto	Apoyo
PATCH	/products/{id}/status	Cambia a `SOLD_OUT` o `AVAILABLE`	CF-MENU-0003
PATCH	/products/{id}/price	Cambia el precio	CF-MENU-0004
PUT	/products/{id}/stock	Registra las unidades del día	CF-MENU-0005
POST	/products/reserve	Uso interno de orders-ms: verifica disponibilidad, descuenta unidades y devuelve precios; si un producto falla, no descuenta nada	CF-PEDI-0001, CF-MENU-0005
POST	/products/release	Uso interno de orders-ms: devuelve unidades de un pedido cancelado	CF-MENU-0005
GET	/health	Verifica que el servicio está vivo	Apoyo

8.2 orders-ms (puerto interno 8000)

Método	Ruta	Qué hace	Historia
GET	/slots	Franjas de hoy con su estado: disponible, pasada o llena	CF-PEDI-0003, 0009
POST	/orders	Crea el pedido `{customer_name, student_code, slot, items: [{product_id, quantity}]}` y devuelve su número	CF-PEDI-0001, 0002, 0003

Método	Ruta	Qué hace	Historia
GET	/orders?status=RECEIVED	Pendientes del día ordenados por franja	CF-PEDI-0004
GET	/orders?status=READY	Pedidos para entregar	CF-PEDI-0005
GET	/orders/{number}	Estado y franja de un pedido del día	CF-PEDI-0006
POST	/orders/{number}/ready	Marca el pedido como listo	CF-PEDI-0005
POST	/orders/{number}/deliver	Registra la entrega `{payment_method}`	CF-PEDI-0007
POST	/orders/{number}/cancel	Cancela con `{student_code}` del estudiante; devuelve las unidades a menu-ms	CF-PEDI-0008
GET / PUT	/config	Consulta o cambia `slot_capacity`, `start_time` y `end_time`	CF-PEDI-0009
GET	/sales/today	Total vendido hoy por medio de pago	CF-PEDI-0007
GET	/health	Verifica que el servicio está vivo	Apoyo

Para cancelar se pide el código estudiantil además del número, para que nadie cancele un pedido ajeno solo adivinando el consecutivo.

8.3 Ejemplo: crear un pedido

```
POST /orders
{
  "customer_name": "Michell Caicedo",
  "student_code": "461224000",
  "slot": "12:15",
  "items": [
    {"product_id": 1, "quantity": 1},
    {"product_id": 4, "quantity": 2}
  ]
}

201 Created
{
  "number": "007",
  "status": "RECEIVED",
  "slot": "12:15",
  "total": 17000
}
```

9. Diagrama de secuencia: hacer un pedido

Hacer un pedido es el único flujo en que participan los dos microservicios, por eso es el que se detalla. Las peticiones del estudiante pasan por nginx, que aquí se omite para simplificar.

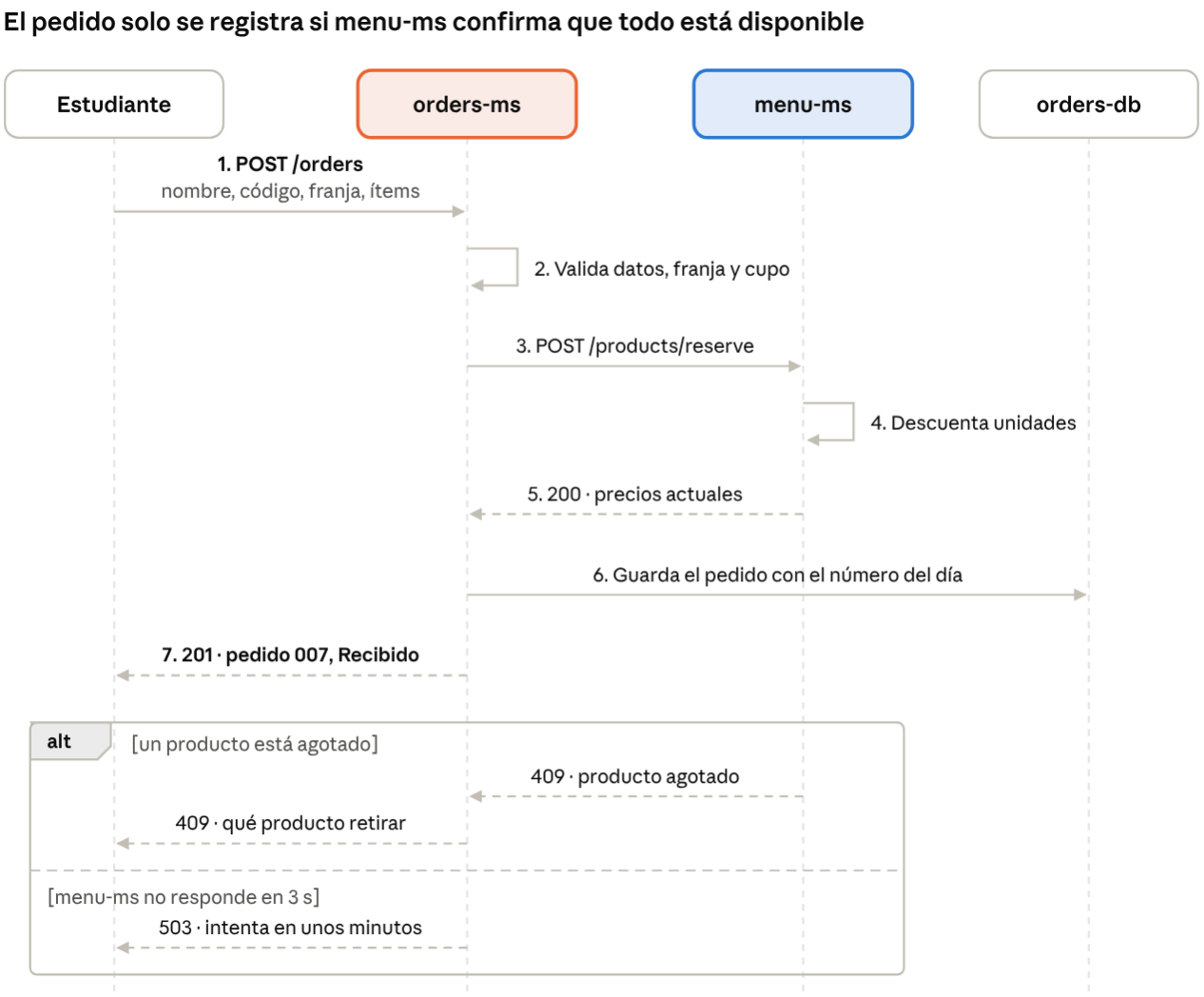


Diagrama de secuencia · hacer un pedido (CF-PEDI-0001 y 0002)

Los pasos 2 a 6 se ejecutan dentro de una transacción de PostgreSQL con un bloqueo por día (pg_advisory_xact_lock), que funciona aunque haya 3 réplicas de orders-ms; un candado dentro del programa no serviría, porque cada réplica tendría el suyo. Así, si dos estudiantes confirman al mismo tiempo, uno espera unos milisegundos al otro: nunca reciben el mismo número, ni se pasa el cupo de la franja, ni se venden más unidades de las que hay (CF-PEDI-0002, escenario 3). Si menu-ms rechaza la reserva, no se descontó nada y el pedido no se guarda.

10. Diagrama de estados del pedido

Un pedido pasa por cuatro estados y solo avanza en un sentido. Cada cambio de estado lo hace un endpoint de orders-ms, que rechaza cualquier cambio no permitido con el mensaje del criterio de aceptación.

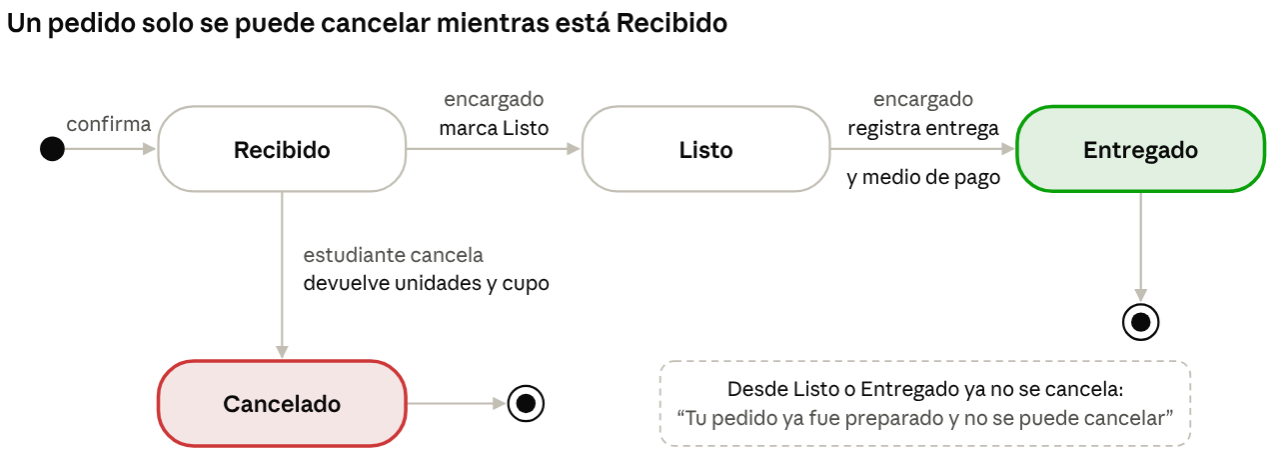


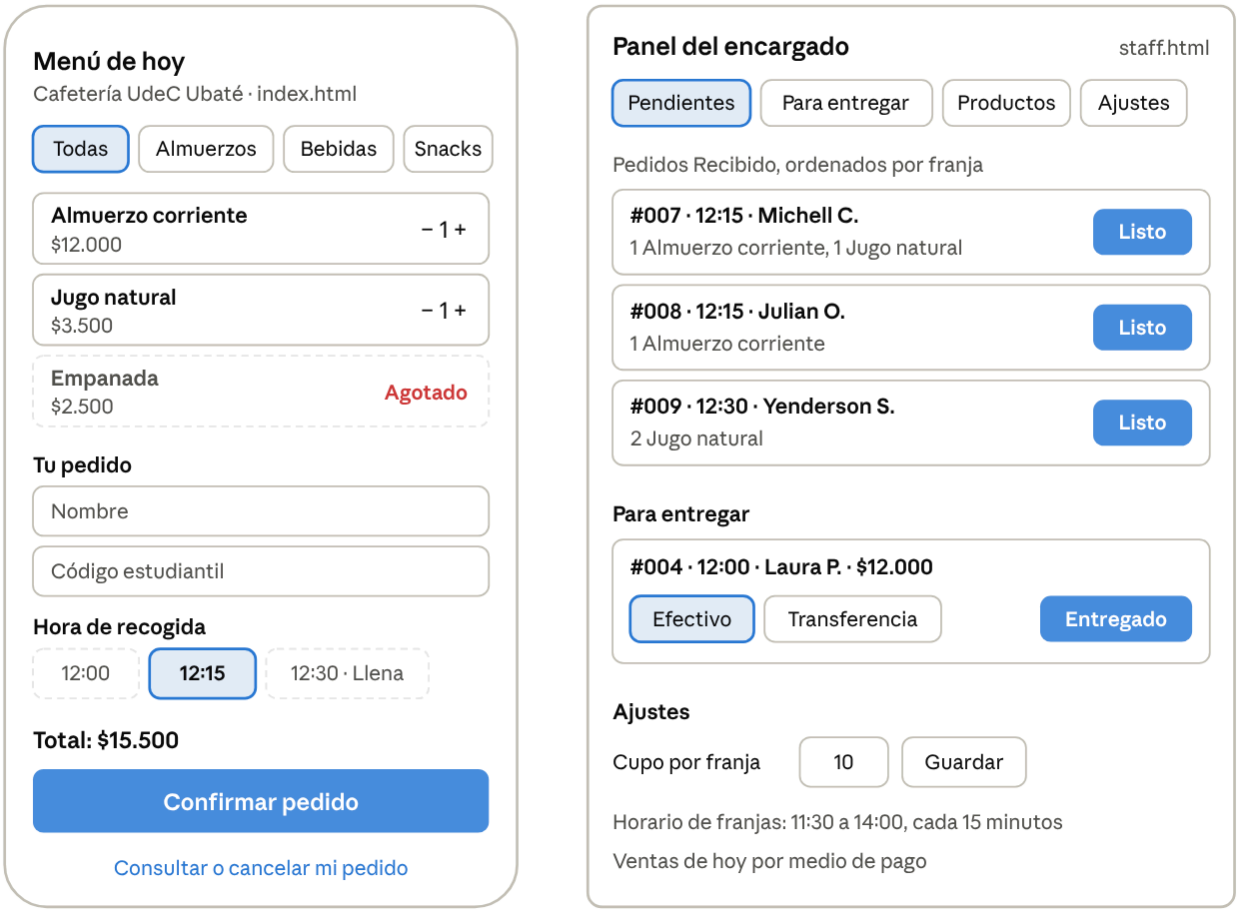
Diagrama de estados del pedido · 4 estados

Intento no permitido	Respuesta del sistema	Historia
Marcar Listo un pedido Cancelado	"Este pedido fue cancelado"	CF-PEDI-0005
Entregar un pedido Recibido	"Primero marca el pedido como Listo"	CF-PEDI-0007
Entregar sin elegir medio de pago	"Selecciona el medio de pago"	CF-PEDI-0007
Cancelar un pedido Listo o Entregado	"Tu pedido ya fue preparado y no se puede cancelar"	CF-PEDI-0008

11. Bocetos de pantallas

El sistema tiene dos páginas: `index.html` para el estudiante, pensada para el celular, e `staff.html` para el computador de la caja. Los nombres y precios del boceto son de ejemplo.

El estudiante pide desde el celular; el encargado atiende en la caja



Bocetos de pantallas · estudiante (celular) y encargado (caja); nombres y precios de ejemplo

En la página del estudiante, los productos agotados y las franjas pasadas o llenas aparecen apagados y no se pueden elegir (CF-MENU-0002, CF-PEDI-0003 y 0009). En el panel del encargado, cada pestaña es una vista: el boceto las muestra juntas. La lista de pendientes se actualiza sola cada 10 segundos para que no tenga que recargar la página mientras atiende.

12. Plan de pruebas

Cada escenario de los criterios de aceptación se convierte en un caso de prueba: las 15 historias suman 46 escenarios. Las pruebas se hacen en tres niveles.

Nivel	Qué se prueba	Herramienta	Quién / cuándo
Unitarias / de API	Cada escenario de cada historia, por microservicio	pytest + TestClient de FastAPI	Al terminar cada historia, antes de hacer commit

Nivel	Qué se prueba	Herramienta	Quién / cuándo
Integración	orders-ms llamando a menu-ms dentro de Docker (pedido, agotado, cancelación, menu-ms caído)	Postman o script con httpx	Al cerrar cada sprint
Aceptación	Flujo completo en la página: publicar menú, pedir, preparar, entregar	Prueba manual desde el celular	Antes de la sustentación, con 3 compañeros

Casos de prueba principales

ID	Historia y escenario	Entrada	Resultado esperado
CP-01	CF-MENU-0001 esc. 1	Almuerzo corriente, almuerzo, 12000	201 y estado Disponible
CP-02	CF-MENU-0001 esc. 3	Precio 0, -500 o "abc"	400 "El precio debe ser un número mayor a 0"
CP-03	CF-MENU-0001 esc. 4	Nombre repetido con otras mayúsculas	409 "Ya existe un producto con ese nombre"
CP-04	CF-MENU-0002 esc. 3	Menú sin productos	"La cafetería aún no ha publicado el menú de hoy"
CP-05	CF-PEDI-0001 esc. 2	Pedido sin productos	400 "Agrega al menos un producto"
CP-06	CF-PEDI-0001 esc. 4	Contenedor menu-ms detenido	503 "No fue posible verificar el menú, intenta de nuevo en unos minutos"
CP-07	CF-PEDI-0002 esc. 3	20 pedidos enviados al mismo tiempo	20 números distintos, del 001 al 020
CP-08	CF-MENU-0004 esc. 3	Cambiar precio después de un pedido	El pedido conserva el precio anterior
CP-09	CF-PEDI-0003 esc. 2	Franja que empieza en 10 minutos	Aparece deshabilitada
CP-10	CF-PEDI-0007 esc. 3	Entregar un pedido en estado Recibido	409 "Primero marca el pedido como Listo"

ID	Historia y escenario	Entrada	Resultado esperado
CP-11	CF-PEDI-0008 esc. 3	Cancelar un pedido Listo	409 "Tu pedido ya fue preparado y no se puede cancelar"
CP-12	CF-MENU-0005 esc. 3 y 4	3 unidades, pedir 3, luego cancelar 1	Agotado al llegar a 0; Disponible con 1 al cancelar
CP-13	CF-PEDI-0009 esc. 2 y 3	Cupo 2, hacer 2 pedidos en la misma franja, cancelar 1	Franja "Llena", luego habilitada otra vez

La lista completa de los 46 casos se genera junto con el código, un archivo de prueba por microservicio (`tests/`), y se ejecuta con `pytest` .

Además, en Docker Swarm se prueba la tolerancia a fallas: con el sistema en uso se detiene una réplica de orders-ms y se verifica que los pedidos siguen funcionando y que Swarm crea una réplica nueva; también se publica una versión nueva con `docker service update` sin cortar el servicio.

13. Despliegue con Docker y control de versiones con Git

Docker se usa en tres niveles. Cada microservicio tiene un **Dockerfile** que construye su imagen. Para desarrollar, **Docker Compose** levanta todo en un equipo con `docker compose up --build` (página en `http://localhost:8080`). Para producción, las imágenes se publican en un registro (GitHub Container Registry, porque `docker stack deploy` no construye imágenes) y el archivo `docker-stack.yml` se despliega en **Docker Swarm**:

```
docker swarm init                                    # crea el clúster (nodo
manager)
printf 'clave' | docker secret create menu_db_pass -
printf 'clave' | docker secret create orders_db_pass -
docker stack deploy -c docker-stack.yml lunch        # despliega el stack
docker service ls                                    # ver servicios y
réplicas
docker service scale lunch_orders-ms=5               # escalar en la hora pico
```

13.1 Servicios en Docker Swarm

Servicio	Imagen	Réplicas	Configuración
web	nginx:alpine + páginas	2	Puerto 8080 en el routing mesh
menu-ms	propia (Dockerfile)	2	Healthcheck `/health`; rolling update
orders-ms	propia (Dockerfile)	3	Healthcheck `/health`; rolling update; se escala en la hora pico
menu-db	postgres:16	1	Volumen `menu-data`; solo en el manager; contraseña con docker secret
orders-db	postgres:16	1	Volumen `orders-data`; solo en el manager; contraseña con docker secret

Los servicios se encuentran por nombre en la red overlay del stack (`MENU_URL=http://menu-ms:8000`) y Swarm reparte las peticiones entre réplicas. Las actualizaciones se hacen réplica por réplica, iniciando la nueva antes de detener la vieja (`order: start-first`), y la zona horaria es `America/Bogota` para que las franjas usen la hora de Colombia. Resumen del `docker-stack.yml`:

```
services:
  web:
    image: ghcr.io/<usuario>/lunch-web:1.0
    ports: ["8080:80"]
    deploy: { replicas: 2 }
  menu-ms:
    image: ghcr.io/<usuario>/menu-ms:1.0
    environment: { DB_HOST: menu-db, TZ: America/Bogota }
    deploy:
      replicas: 2
      update_config: { parallelism: 1, order: start-first }
      restart_policy: { condition: on-failure }
  orders-ms:
    image: ghcr.io/<usuario>/orders-ms:1.0
    environment: { DB_HOST: orders-db, MENU_URL: "http://menu-ms:8000", TZ:
America/Bogota }
    deploy:
      replicas: 3
      update_config: { parallelism: 1, order: start-first }
      restart_policy: { condition: on-failure }
  menu-db:
    image: postgres:16
```

```
    environment: { POSTGRES_PASSWORD_FILE: /run/secrets/menu_db_pass }
    secrets: [menu_db_pass]
    volumes: ["menu-data:/var/lib/postgresql/data"]
    deploy: { placement: { constraints: ["node.role == manager"] } }
  # orders-db se configura igual que menu-db
volumes: { menu-data: {}, orders-data: {} }
secrets:
  menu_db_pass: { external: true }
  orders_db_pass: { external: true }
```

13.2 Estructura del repositorio

```
lunch-on-time/
├── docker-compose.yml   (desarrollo)
├── docker-stack.yml     (producción en Docker Swarm)
├── README.md
├── docs/                (documentación del proyecto)
├── menu-ms/
|   ├── Dockerfile
|   ├── requirements.txt
|   ├── app/  (main.py, db.py)
|   └── tests/  (test_menu.py)
├── orders-ms/
|   ├── Dockerfile
|   ├── requirements.txt
|   ├── app/  (main.py, db.py, menu_client.py)
|   └── tests/  (test_orders.py)
└── web/
    ├── Dockerfile      (nginx con las páginas)
    ├── nginx.conf
    ├── index.html      (estudiante)
    └── staff.html      (encargado)
```

13.3 Flujo de trabajo con Git

1. **Convención de idioma:** todo lo técnico va en inglés (código, variables, funciones, archivos, microservicios, endpoints, tablas, campos, ramas y commits). Los textos que ve el usuario (botones y mensajes de error) quedan en español, con las frases exactas de los criterios de aceptación.
2. Un repositorio en GitHub, `lunch-on-time`, con la rama `main` protegida: solo recibe código probado.
3. Una rama por historia de usuario, con su ID en el nombre: `feature/CF-MENU-0001-register-product`.

4. Commits pequeños en inglés con el formato **Conventional Commits**: `feat(menu): register product [CF-MENU-0001]`, `fix(orders): reject empty order`, `test(orders): add concurrency test`, `docs: update README`.
5. Al terminar la historia y pasar sus pruebas, se abre un *pull request* a `main` y otro integrante lo revisa.
6. Al cerrar cada sprint se marca una etiqueta (`v0.1-sprint1` … `v0.5-sprint5`).

14. Uso de inteligencia artificial en el proyecto

Por indicación del docente, el proyecto usa IA en todas las etapas; el equipo revisa, prueba y responde por cada resultado.

Etapa	Herramienta	Para qué se usa	Qué revisa el equipo
Diagnóstico	Google Forms + Claude	Diseño de la encuesta y síntesis de resultados	Que las conclusiones salgan de los datos
Análisis de datos	Google Colab (Python con pandas y matplotlib, y su asistente de IA)	Tablas y gráficas de la encuesta	Que los porcentajes coincidan con las respuestas
Análisis y diseño	Claude	Este documento: requisitos, arquitectura, modelo de datos, APIs y diagramas	Que cumpla las historias y lo pedido por el docente
Desarrollo	Google Antigravity (agente con el modelo elegido en el selector)	Generar el código de los microservicios, Dockerfiles, docker-compose y pruebas a partir de este documento	Que pasen las pruebas y que el equipo pueda explicar el código
Pruebas	Antigravity + pytest	Generar los 46 casos de prueba y corregir fallos	Que cada caso corresponda a un escenario real

14.1 Prompt base para Antigravity

El desarrollo en Antigravity se hace historia por historia, en el orden de los sprints. Para empezar, se abre la carpeta del repositorio como proyecto, se adjuntan este documento y el Excel de historias de usuario, y se envía este prompt:

```
Eres el desarrollador del proyecto "Almuerzo a Tiempo" (repositorio lunch-on-
time):
sistema de pedidos anticipados de la Cafetería UdeC Ubaté. La documentación
está en la
carpeta docs/ (documento de análisis y diseño, historias de usuario y
artículo).

Convención de idioma: todo lo técnico en inglés (código, variables,
funciones,
archivos, endpoints, tablas, campos, ramas y commits con Conventional
Commits).
Los textos que ve el usuario (botones y mensajes de error) en español, con
las
frases exactas de los criterios de aceptación.

Construye el sistema exactamente como dice el documento:
- Dos microservicios en Python 3.12 + FastAPI: menu-ms y orders-ms, cada uno
con su
  propia base PostgreSQL, con las tablas, campos y endpoints de las secciones
7 y 8.
- orders-ms se comunica con menu-ms por REST (httpx) usando MENU_URL.
- Como orders-ms corre con 3 réplicas, la exclusión mutua va en la base de
datos:
  pg_advisory_xact_lock por día para el número y el cupo, y UPDATE
condicional para
  descontar stock. Nada de candados en memoria.
- Si guardar el pedido falla después de reservar, libera las unidades
(compensación).
- Una página web (index.html para el estudiante y staff.html para el
encargado)
  servida por nginx, que reenvía /api/menu y /api/orders a cada
microservicio.
- Un Dockerfile por servicio, docker-compose.yml para desarrollo y docker-
stack.yml
  para Docker Swarm como en la sección 13 (réplicas, healthcheck, secrets).
- Pruebas con pytest (test_menu.py y test_orders.py) para cada escenario de
cada
  historia, más una prueba de concurrencia con pedidos simultáneos.

Trabaja por sprints. Empieza con el Sprint 1: crea menu-ms con PostgreSQL
(CF-MENU-0001, 0002 y 0003), su Dockerfile y el docker-compose.yml. Antes de
escribir código, muéstrame tu plan. Al terminar, corre las pruebas y
explícame en
español, con palabras sencillas, qué hace cada archivo.
```

Al terminar cada sprint se pide: "Continúa con el Sprint N", se revisan las pruebas y se hace commit con el ID de la historia.

Referencias

Caicedo, M. D., Ordoñez, J. E., & Silva Gomez, Y. J. (2026). *Diagnóstico binario: árbol de problemas del servicio de cafetería de la Universidad de Cundinamarca, seccional Ubaté* [Trabajo de clase, Sistemas Operativos]. Universidad de Cundinamarca.

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Docker Inc. (s. f.). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Ramírez, S. (s. f.). *FastAPI*. https://fastapi.tiangolo.com/es/

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.